SINAL

Etapa 4: Arquitetura Técnica

Sistema de emergência acessível, visual e direto para pessoas surdas — MVP acadêmico

Campo	Valor
Documento	04-arquitetura-tecnica-v1.pdf
Título	SINAL — Etapa 4: Arquitetura Técnica
Versão	v1
Status	**APROVADA**
Data	06/10/2026
Documentos de base	Etapa 1 (Reinicialização Conceitual), Etapa 2 (Requisitos e Fluxos), Etapa 3 (UX/UI e Figma), Etapa 3.1 (Registro do Protótipo)
Próxima etapa	Etapa 5 — Modelo de Domínio (após validação deste documento)

Documento cumulativo: reúne a arquitetura técnica aprovada, suas decisões, alternativas descartadas, decisões adiadas, riscos e critérios de aceitação, de forma que quem não acompanhou as etapas anteriores consiga compreendê-la. Não contém código.

Índice

1. Objetivo da Etapa 4	**18.** Simulação
2. Contexto arquitetural do SINAL	**19.** Jurisdição
3. Stack tecnológica	**20.** Segurança
4. Princípios arquiteturais	**21.** Frontend
5. Arquitetura geral	**22.** Android futuro
6. Estrutura de módulos	**23.** LGPD
7. Responsabilidade das camadas	**24.** Observabilidade
8. Domain	**25.** Testes
9. Application	**26.** Microsserviços e evolução futura
10. Presentation	**27.** Decisões aprovadas
11. Infrastructure	**28.** Decisões descartadas
12. Persistência	**29.** Decisões adiadas
13. Repositórios e adapters	**30.** Riscos
14. Modelo de módulos e dependências	**31.** Critérios de aceitação
15. Fluxos arquiteturais	**32.** Validação final da Etapa 4
16. API REST	**33.** Próxima etapa
17. Idempotência	

*Legenda de status usada neste documento: **Aprovada** = decisão tomada nas etapas anteriores ou nesta etapa. **Adiada** = fica para a Etapa 5, para a implementação ou para etapas futuras.*

1. Objetivo da Etapa 4

Transformar os requisitos, fluxos e decisões de UX/UI já aprovados (Etapas 1 a 3.1) em uma arquitetura técnica consistente, evolutiva e preparada para implementação. A arquitetura deve:

- permitir implementar o MVP Web, mantendo backend em Java + Spring Boot, MySQL e frontend React + TypeScript;
- manter as regras de negócio independentes de detalhes de infraestrutura;
- ser testável, segura e evolutiva;
- permitir substituir a simulação por integrações reais sem reescrever o domínio;
- permitir futuramente um cliente Android consumindo a mesma API;
- não introduzir complexidade arquitetural desnecessária.

Esta etapa é de **definição arquitetural**. Não há código Java, React, SQL, migrations ou implementação de qualquer natureza.

2. Contexto arquitetural do SINAL

SINAL é um sistema de emergência acessível, visual e direto, desenvolvido como MVP acadêmico para pessoas surdas, que permite solicitar ajuda sem depender de voz ou ligação telefônica. Os serviços são Polícia Militar, SAMU, Corpo de Bombeiros e contato de confiança, além da **Emergência Imediata**, atalho de pânico direcionado à Polícia Militar.

O encaminhamento aos órgãos é **simulado**: não há integração real com PM, SAMU ou Bombeiros. O sistema nunca apresenta como fato que uma central recebeu a solicitação, que uma equipe foi acionada ou está a caminho, nem usa o termo “Em Atendimento”.

O aviso ao contato de confiança é um fluxo paralelo, **não simulado**, que usa o compartilhamento nativo do dispositivo (SMS, WhatsApp, Telegram). O SINAL nunca afirma que a mensagem foi enviada, entregue, recebida ou lida.

Premissas

- Um único deployable de backend, um SPA e um banco MySQL.
- Toda solicitação pertence a um Usuario autenticado (fluxo Splash → Login/Cadastro → Home).
- O frontend apresenta e captura. O backend decide as regras.
- A geolocalização é capturada no dispositivo, o que exige HTTPS também em homologação.
- Localização negada ou indisponível bloqueia o envio. Somente a baixa precisão não bloqueia.
- “Avisar contato” não gera Chamado nem item de Histórico.
- Terceiro reportando em nome de outra pessoa, recuperação de senha e cancelamento de solicitação estão fora do MVP.

3. Stack tecnológica

Camada	Tecnologia	Observação
Backend	Java, Spring Boot, API REST	Monólito modular. Spring Security, Spring Data JPA, Actuator/Micrometer
Banco de dados	MySQL 8, Flyway	Migrações versionadas (implementação). `utf8mb4`
Frontend	React, TypeScript, Vite, React Router, TanStack Query	Web responsiva, mobile-first. Estrutura por features
Futuro	Android híbrido (Capacitor ou equivalente)	Outro cliente da **mesma API**. Etapa 9

4. Princípios arquiteturais

O princípio fundamental é “preparar para evoluir sem construir hoje o que ainda não é necessário”. A pergunta que orienta cada decisão é:

“Essa separação existe porque há uma responsabilidade arquitetural real ou apenas porque uma arquitetura recomenda?”

Uma abstração (interface, port, adapter, mapper) só existe quando há ganho real em ao menos um destes critérios:

- **Substituição real** à vista (simulação por integração, jurisdição mock por real);
- **Isolamento do domínio** de framework, onde o ganho é concreto (agregados com comportamento);
- **Testabilidade** que, de outro modo, exigiria subir infraestrutura.

Distinção central de responsabilidades

Tipo	O que é	Exemplo no SINAL	Onde mora
Regra de negócio	Verdade válida independentemente de HTTP, banco ou framework	EI é sempre PM e não tem ocorrência; só se vai de CRIADO a ENCAMINHADO_SIMULADO e depois ENCERRADO; latitude entre -90 e 90; no máximo 5 contatos	**Domain**
Orquestração	Sequência de passos que coordena regras e recursos	Verificar idempotência, carregar a ocorrência, chamar `Chamado.registrar`, salvar, auditar, publicar evento	**Application**
Infraestrutura	Como algo é gravado, lido ou enviado	JPA, SQL, JWT, agendador, chamada HTTP externa futura	**Infrastructure**
Entrada e saída HTTP	Contrato com o cliente	Controllers, DTOs, validação sintática, problem+json	**Presentation**

Critério prático: se trocar o framework ou o banco não deveria alterar a regra, ela é de domínio. Se a regra é “o que fazer em que ordem”, é orquestração.

A regra geral é **arquitetura limpa e evolutiva, mas pragmática**. Não se cria interface por serviço "porque toda classe tem interface", nem mapper entre classes idênticas, nem camada ou subpasta sem conteúdo. O Spring é usado normalmente onde pertence.

5. Arquitetura geral

Monólito modular, em camadas, com princípios de DDD e Ports and Adapters aplicados somente onde há fronteira real. Em cada módulo, as camadas Presentation, Application, Domain e Infrastructure têm a seguinte dependência conceitual:

```
Presentation
     ↓
Application
     ↓
  Domain
     ↑
Infrastructure
```

O Domain é Java puro e não depende de Spring, Spring Data, JPA, Hibernate, HTTP, JSON, DTOs, banco de dados ou APIs externas. A Infrastructure implementa os contratos definidos pelo Domain quando esses contratos representam fronteiras de negócio.

Diagrama de contexto

```
Pessoa surda ──────► [ SINAL ] ──(app externo no dispositivo)──► Contato de confiança
(usuário)               │
                        ┆  (tracejado = NÃO integrado no MVP)
                        ▼
            PM · SAMU · Corpo de Bombeiros
```

Diagrama de containers

```
┌──────────────────┐   HTTPS / JSON   ┌──────────────────────────┐      ┌─────────┐
│ React SPA (Web)  │ ───────────────► │ Spring Boot              │ ───► │  MySQL  │
│ + futuro Android │     /api/v1      │ monólito modular         │      └─────────┘
│   (Capacitor)    │                  │ (1 deployable)           │
└──────────────────┘                  └──────────────────────────┘
  GPS e compartilhamento: via dispositivo
```

Diagrama de camadas (por módulo)

```
presentation ──► application ──► domain ◄── infrastructure
(HTTP, DTOs)     (casos de uso,   (agregados,    (JPA, adapters, segurança,
                  transação,       VOs, regras,   simulação, mock, config)
                  eventos)         contratos)
```

6. Estrutura de módulos

A estrutura é por **contexto** (pacote raiz `br.sinal`). Dentro de cada módulo, aplicam-se **somente as camadas que possuem responsabilidade real**. Não se cria estrutura vazia para manter simetria.

```
br.sinal
├── compartilhado/
│   └── domain/
│
├── identidade/
│   ├── presentation/
│   ├── application/
│   ├── domain/
│   └── infrastructure/
│
├── chamado/
│   ├── presentation/
│   ├── application/
│   ├── domain/
│   └── infrastructure/
│
├── contato/
│   ├── presentation/
│   ├── application/
│   ├── domain/
│   └── infrastructure/
│
├── jurisdicao/
│   ├── domain/
│   └── infrastructure/
│
├── encaminhamento/
│   ├── application/
│   ├── domain/
│   └── infrastructure/
│
└── plataforma/
```

Organização interna (conceitual)

```
identidade/
  presentation/    AuthController, UsuarioController, DTOs
  application/     cadastro, login, renovação de token
  domain/          model/ (Usuario)   repository/ (UsuarioRepository)
  infrastructure/  persistence/ (entidades JPA, Spring Data, adapter)
                   security/ (JWT, filtros, encoder)

chamado/
  presentation/    ChamadoController, OcorrenciaController, DTOs
  application/     AberturaDeChamadoService, consultas (Histórico, Detalhe, Acompanhamento)
  domain/          model/ (Chamado, EstadoChamado, ModalidadeChamado, Transicao)
                   repository/ (ChamadoRepository)
  infrastructure/  persistence/ (entidades JPA, Spring Data, adapter, projeções)

contato/
  presentation/    ContatoController, DTOs
  application/     casos de uso de contatos e do registro de compartilhamento
  domain/          ContatosDeConfianca (agregado), repository
  infrastructure/  persistence/ (somente persistência)

jurisdicao/        domain/ (Jurisdicao, JurisdicaoGateway)   infrastructure/ (JurisdicaoMockAdapter)

encaminhamento/    application/ (EncaminhamentoService, DeterminacaoDeCentralService, job)
                   domain/ (Encaminhamento, EncaminhamentoGateway, repository)
                   infrastructure/ (persistence/, simulacao/EncaminhamentoSimuladoAdapter)

plataforma/        config, segurança HTTP, tratamento de erros (problem+json),
                   correlação, auditoria, observabilidade, Clock

compartilhado/     domain/ (OrgaoSeguranca, Localizacao)
```

Convenção de pacotes: `domain/model`, `domain/repository`, `application`, `presentation`, `infrastructure/persistence`. *Nomes finais de classes e sufixos (Port/Gateway/UseCase/Service) ficam para a implementação.*

Por que nem todo módulo tem as quatro camadas completas

Módulo	Camadas	Justificativa
`identidade`	4	Entrada HTTP própria, segurança, persistência e regras de cadastro.
`chamado`	4	Núcleo do domínio, com entrada HTTP, persistência e consultas próprias.
`contato`	4 (infraestrutura apenas de persistência)	Tem HTTP e persistência. O domínio é pequeno, e nada além do necessário é criado.
`jurisdicao`	domain + infrastructure	Sem HTTP e sem casos de uso próprios: é um contrato e uma implementação mock.
`encaminhamento`	application + domain + infrastructure	Sem endpoint próprio: reage a eventos e a um job. O polling do cliente é servido pelo módulo `chamado`.
`compartilhado`	domain	Somente tipos de domínio realmente compartilhados (`OrgaoSeguranca`, `Localizacao`).
`plataforma`	transversal	Configuração, segurança HTTP, erros, correlação, auditoria, observabilidade e `Clock`.

7. Responsabilidade das camadas

Camada	Responsabilidade	Não faz
Presentation	Entrada e saída HTTP. DTOs de request e response. Validação **sintática**. Extrai o usuário autenticado. Converte DTO em comando e chama **um** caso de uso.	Regra de negócio. Acesso a repositório ou JPA. Decidir transição de Chamado. Implementar simulação.
Application	Orquestra o caso de uso: transação, verificação de propriedade, idempotência, chamada ao domínio e aos ports, publicação de eventos, auditoria.	Regra que pertence ao agregado.
Domain	Agregados, entidades de domínio, Value Objects, enums, **invariantes**, transições válidas e contratos de fronteira de negócio.	Spring, JPA, Hibernate, HTTP, JSON, DTO, banco.
Infrastructure	JPA, Spring Data, entidades de persistência, adapters, segurança técnica, simulação, mock de jurisdição, configuração, observabilidade.	Regra de negócio.

8. Domain

- **Java puro.** Contém `Chamado` (Aggregate Root), `Localizacao` (Value Object imutável), `Usuario`, `ContatosDeConfianca`, `Encaminhamento`, enums, invariantes, regras e transições válidas.
- **Não é anêmico.** O `Chamado` expõe operações com sentido de negócio (registrar, registrar Emergência Imediata, avançar para encaminhado, encerrar) e valida as invariantes.
- **Contratos de fronteira de negócio ficam no Domain** (decisão K1): `ChamadoRepository`, `UsuarioRepository`, `JurisdicaoGateway` e `EncaminhamentoGateway`. O domínio conhece o contrato, não a implementação.
- **Contratos puramente técnicos** (publicador de métricas, provedor de JWT) **não** vão para o domínio: ficam na application ou na infrastructure.
- Referências entre agregados são **por id** (`Long`), sem navegação de objetos entre agregados ou módulos.

```
Domain
  EncaminhamentoGateway
          ▲
          | implementação
          |
Infrastructure
  EncaminhamentoSimuladoAdapter
```

Chamado (visão arquitetural; a modelagem detalhada é da Etapa 5)

- **Aggregate Root** do domínio de emergência. Identidade `Long`; `usuarioId` por referência.
- Modalidade: padrão ou Emergência Imediata, no **mesmo agregado**, com factories distintas. O comportamento próprio (órgão fixo PM, sem ocorrência) vive nas factories e na invariante.
- Invariantes: EI implica PM e sem ocorrência; modalidade padrão exige órgão e ocorrência válida (inclusive a genérica); localização válida; detalhe opcional e limitado; transições só pelos métodos do agregado.
- Localização é Value Object imutável pertencente ao Chamado (e não ao Usuario): latitude, longitude, precisão e timestamp (`Instant`). É obrigatória nos fluxos de emergência.
- Cada Chamado tem estado próprio. **Não existe "chamado ativo do usuário"**: múltiplos Chamados podem estar em andamento simultaneamente.

Estados do Chamado

Estado interno	Texto exibido ao usuário
`CRIADO`	Solicitação registrada
`ENCAMINHADO_SIMULADO`	Simulação de encaminhamento em andamento
`ENCERRADO`	Simulação encerrada

- Transições válidas: `CRIADO → ENCAMINHADO_SIMULADO → ENCERRADO`. `ENCERRADO` é terminal. Transição inválida é rejeitada pelo agregado e tratada como no-op idempotente quando repetida.
- O backend devolve o **código** do estado e `modoEncaminhamento: SIMULADO`. O frontend mapeia para os textos aprovados.
- **Não existem** os estados "Em Atendimento" nem `CANCELADO` no MVP.
- **Evolução futura:** estados reais poderão ser adicionados quando houver integração, vinculados ao modo `REAL`. O frontend trata código desconhecido com fallback neutro. Nada que sugira atendimento real pode ser emitido enquanto o único adapter for o simulado.

Histórico de estados e eventos

- Tabela append-only de transições (`chamado_transicao`): alimenta timestamps por estado, acompanhamento e auditoria.
- Eventos internos (`ChamadoRegistrado`, `EstadoAlterado`) publicados por Spring Events **após o commit**, para desacoplar o módulo `encaminhamento`. Sem broker e sem event sourcing.

9. Application

A Application orquestra os casos de uso. Pode usar Spring (`@Service`, `@Transactional`, eventos, `@Scheduled`) e declara os ports de saída puramente técnicos. **Não concentra regras de negócio** que pertencem ao agregado.

Serviço de aplicação	Papel	Observação
`AberturaDeChamadoService`	Registra o Chamado (normal ou EI), com idempotência, verificação de propriedade, auditoria e evento.	Um serviço, dois pontos de entrada HTTP.
`EncaminhamentoService`	Cria o Encaminhamento e atualiza o estado do chamado.	Acionado pelo evento e pelo job.
`DeterminacaoDeCentralService`	Consulta a jurisdição pelo `JurisdicaoGateway` e resolve a central.	Orquestra a consulta ao gateway.

Decisão K2: `EncaminhamentoService` e `DeterminacaoDeCentralService` são **Application Services** e fazem orquestração. Se a Etapa 5 identificar uma regra pura de negócio que realmente mereça um Domain Service, isso será decidido na Etapa 5. Nenhum Domain Service é criado artificialmente nesta etapa.

Além do que está descrito acima, a Application realiza: verificação de propriedade (`usuarioId` vem do token), resolução de idempotência, auditoria e publicação do evento `ChamadoRegistrado` após o commit. **Não há interface por serviço de aplicação.**

10. Presentation

- Controllers REST, DTOs de request e response, validação sintática (Bean Validation), tratamento de entrada HTTP e conversão para comandos de aplicação.
- Controllers **não** contêm regra de negócio, **não** acessam Repository, **não** manipulam JPA, **não** decidem transições de Chamado e **não** implementam a simulação.
- **Sem mass assignment.** Os DTOs de entrada não possuem como campos do corpo: `id`, `usuarioId`, `estado`, `modalidade` ou `chaveIdempotencia`. A chave vem **somente** do header `Idempotency-Key`. O `usuarioId` vem **somente** do token. O estado só é alterado pelo domínio.
- Um único `@RestControllerAdvice` (em `plataforma`) converte exceções em `application/problem+json`. Nunca sai stack trace pela API.

Validação em três níveis

Nível	Onde	Exemplos
Sintática	Presentation	Formato, tamanho, obrigatoriedade
Aplicação	Application	Propriedade, existência, ocorrência ativa e do órgão certo, limite de 5 contatos, idempotência
Domínio	Domain	Faixa de latitude e longitude, EI ⇒ PM, transições válidas

11. Infrastructure

Responsável pelos detalhes técnicos: JPA, Spring Data, entidades de persistência, adapters, segurança (JWT, filtros, encoder), configuração, integração de simulação, mock de jurisdição e observabilidade. Implementa os contratos definidos no Domain e na Application. Não contém regra de negócio.

É mantida a separação **Domain Entity ≠ JPA Entity** onde ela contribui para a independência do domínio (ver seção 12). As abstrações e mappers só existem quando há responsabilidade real.

12. Persistência

Spring Data JPA, entidades JPA em `infrastructure/persistence`, repositories Spring Data, adapters quando necessário e Flyway. Identificadores `Long` com auto-incremento; `Instant` persistido em UTC (`DATETIME(6)`); enums persistidos por **nome** (nunca ordinal); `utf8mb4`; CPF só com dígitos; e-mail em minúsculas; telefone normalizado.

Onde a separação Domain Entity ≠ JPA Entity agrega valor

Tipo	Separar?	Razão
Chamado	**Sim**	Tem invariantes e máquina de estados e contém um VO que seria `@Embeddable`. Manter o domínio sem anotações protege o ponto mais importante.
Usuario	**Sim**	Protege `senhaHash` e as regras de cadastro.
ContatosDeConfianca	**Sim**	A invariante do limite de 5 pertence ao domínio.
Encaminhamento	**Sim**	O modelo evolui com a integração real.
Catálogo de ocorrências	**Não**	Dado de referência, sem comportamento. Lido por projeção. O Chamado guarda só o `ocorrenciaId`.
Auditoria, refresh token	**Não**	Registros técnicos sem comportamento de domínio.
Consultas de leitura	**Não**	Projeções DTO, sem hidratar o agregado.

Value Object Localizacao

No domínio é imutável e sem anotações (`latitude: Double`, `longitude: Double`, `precisao: Float`, `timestamp: Instant`). Na persistência vira **colunas achatadas** na entidade JPA do Chamado, e o mapper converte. O `@Embeddable` no domínio foi descartado.

Tabelas

Tabela	Responsabilidade
`usuario`	Identidade, `senha_hash`, `ativo`, `data_cadastro`
`refresh_token`	Sessões revogáveis (somente o hash do token)
`ocorrencia_tipo`	Catálogo de ocorrências
`chamado`	Agregado, com localização em colunas achatadas, `baixa_precisao` e `chave_idempotencia`
`chamado_transicao`	Histórico append-only de estados
`encaminhamento`	Modo, status técnico, referências de jurisdição e central, `proxima_transicao_em`
`contato_confianca`	Nome e telefone, por usuário
`auditoria_evento`	Ator, instante, evento e referência. **Obrigatória**

Índices e unicidade

Tabela	Índice / restrição
`usuario`	UNIQUE(cpf), UNIQUE(email)
`chamado`	(`usuario_id`, `data_criacao` DESC, `id`) para o Histórico; UNIQUE(`usuario_id`, `chave_idempotencia`)
`chamado_transicao`	(`chamado_id`, `ocorrido_em`)
`encaminhamento`	(`status`, `proxima_transicao_em`) para o job
`ocorrencia_tipo`	UNIQUE(`codigo`); (`orgao`, `ativo`, `ordem`)
`contato_confianca`	(`usuario_id`)
`refresh_token`	UNIQUE(`token_hash`)
`auditoria_evento`	(`ator_id`, `ocorrido_em`)

Chaves estrangeiras existem no banco, mas o JPA não navega entre módulos: guarda apenas ids.

Catálogo de ocorrências

- Persistido em tabela, **não** é enum: permite evolução, ativação e desativação, ordenação, códigos estáveis e alteração de apresentação sem alterar o domínio.
- Códigos estáveis de negócio (por exemplo `PM_ASSALTO_ROUBO`). Itens já utilizados são imutáveis: mudança = nova linha e desativação da antiga, para o Histórico não mudar retroativamente.
- Seed por migração com as opções aprovadas por órgão. Existe exatamente um item genérico **"Outro / Não sei informar"** por órgão.
- O frontend recebe a lista pela API e mapeia os ícones pelo `codigo`.
- Não existem tabelas para `Jurisdicao` e `CentralAtendimento` no MVP: o `encaminhamento` guarda apenas referências.

13. Repositórios e adapters

O padrão completo é usado **somente onde a separação realmente agrega valor**, e não é regra obrigatória para toda classe persistida:

```
Domain Repository  (interface, no domain/repository)
       ↓
Spring Data Repository  (infrastructure/persistence)
       ↓
Adapter  (traduz agregado ↔ entidade, usa mapper)
       ↓
JPA Entity
```

Caso	Estrutura	Por quê
Agregados (Chamado, Usuario, ContatosDeConfianca, Encaminhamento)	**3 peças**: interface de domínio, Spring Data e adapter com mapper	O adapter traduz entre domínio e JPA. É onde a separação se concretiza.
Consultas de leitura (Histórico, Detalhes, Acompanhamento)	Sem repositório de domínio e sem hidratar o agregado. Interface de consulta na application, implementada na infrastructure com projeções DTO	Consulta não precisa do comportamento do agregado.
Catálogo de ocorrências	Spring Data direto, via interface de consulta	Sem comportamento. Evita adapter que só repassa.
Registros técnicos (auditoria, refresh token)	Spring Data com a abstração mínima necessária	Sem comportamento de domínio.

Teste de necessidade aplicado

Elemento	Existe?	Justificativa
Interface de repositório de domínio (agregados)	Sim	Isola o domínio e permite testar sem banco.
Adapter + mapper de agregados	Sim	Concretiza a separação onde há comportamento.
Repositório de domínio para o catálogo	Não	Sem comportamento.
Repositório de domínio para consultas de tela	Não	Projeção DTO, sem hidratar o agregado.
`JurisdicaoGateway`	Sim	Substituição real.
`EncaminhamentoGateway`	Sim	Substituição real.
Interface por serviço de aplicação	Não	Sem substituição nem ganho de teste.
Mapper DTO ↔ comando	Mínimo	Manual ou MapStruct (adiado para a implementação).
Camada em módulo sem conteúdo	Não	Ver seção 6.

14. Modelo de módulos e dependências

Módulo	Responsabilidade	Fronteira
`identidade`	Usuario, cadastro, login por CPF ou e-mail, tokens	Não conhece chamados.
`chamado`	Chamado, catálogo, Histórico, Detalhes, Acompanhamento	Núcleo. Publica `ChamadoRegistrado`.
`contato`	Contatos de confiança e registro de compartilhamento iniciado	Só persiste e audita. **Não envia.**
`jurisdicao`	`JurisdicaoGateway` e mock	Responde jurisdição para (localização, órgão).
`encaminhamento`	Encaminhamento, simulação, determinação de central	Reage ao evento e atualiza o estado via API do `chamado`.
`compartilhado`	`OrgaoSeguranca`, `Localizacao`	Kernel mínimo.
`plataforma`	Segurança HTTP, erros, correlação, auditoria, observabilidade	Transversal.

```
encaminhamento ──► chamado (API pública)
       └──────────► jurisdicao (API pública)
todos ──► compartilhado
chamado ─ ─ evento ChamadoRegistrado ─ ─► encaminhamento      (sem import)
```

Regras de fronteira

- Um módulo só acessa outro pela sua **API pública** (classes e eventos explicitamente expostos). O restante é package-private onde o Java permitir.
- Referências entre agregados são **por id**.
- `chamado` **não** depende de `encaminhamento` (a ligação é por evento), evitando ciclo.
- Histórico e acompanhamento não são módulos: são o lado de leitura do Chamado, sem regras próprias. `Usuario` fica em `identidade` por ser pequeno e inseparável da autenticação. Localização é Value Object, sem ciclo de vida próprio.

15. Fluxos arquiteturais

Fluxo principal de registro

- O cliente, já com a localização capturada e o payload congelado, chama `POST /api/v1/chamados` ou `POST /api/v1/chamados/emergencia-imediata`, com `Idempotency-Key`.
- A Presentation valida a sintaxe e chama `AberturaDeChamadoService`.
- A Application resolve a idempotência, verifica o usuário do token, valida a ocorrência (se houver), chama o domínio, salva e grava a auditoria **na mesma transação**.
- Após o commit, publica `ChamadoRegistrado`.
- `EncaminhamentoService` reage: `DeterminacaoDeCentralService` consulta o `JurisdicaoGateway`, e cria-se o `Encaminhamento` (modo `SIMULADO`) com a próxima transição agendada.
- O job avança `CRIADO → ENCAMINHADO_SIMULADO → ENCERRADO`, gravando transição e auditoria a cada passo.
- O cliente acompanha por polling.

Fluxo 1 — Solicitação de emergência (SAMU, Bombeiros, PM)

```
Usuário    Frontend                API / chamado             encaminhamento
   │ órgão → ocorrência → Resumo        │                           │
   │──────────►│ captura localização     │                           │
   │           │ gera Idempotency-Key    │                           │
   │           │ "Enviar mensagem" (habilitado: obtida ou baixa precisão)
   │           │ POST /chamados ─────────►│ valida, propriedade,      │
   │           │                         │ idempotência, registra    │
   │           │◄── 201 CRIADO ──────────│ commit                    │
   │           │ tela Solicitação registrada                         │
   │           │                         │── ChamadoRegistrado ─────►│ jurisdição / central
   │           │                         │◄── job: transições ───────│ Encaminhamento SIMULADO
   │           │ Acompanhamento: polling → ENCAMINHADO_SIMULADO → ENCERRADO
```

Fluxo 2 — Emergência Imediata

```
Home → modal "Você está em perigo agora?" → ENVIAR AGORA
  → captura em segundo plano (baixa precisão: aviso ~1 s, segue sozinho, sem 2ª confirmação)
  → POST /chamados/emergencia-imediata  (Idempotency-Key)
  → Chamado(modalidade = EI, órgão = PM, sem ocorrência)       [mesmo agregado]
  → Solicitação registrada → Acompanhamento (mesmo fluxo simulado)
  Cancelar durante a captura: nada é enviado
```

Fluxo 3 — Consulta do Histórico

```
Frontend ── GET /chamados ──► token → usuarioId
         ◄── resumos ──────── consulta escopada ao usuário, paginada;
                              polling de lista enquanto houver item não encerrado
toque no item ── GET /chamados/{id} ──► busca por (id, usuarioId); de outro usuário → 404
```

Fluxo 4 — Acompanhamento

```
Histórico / Detalhes ── "Ver acompanhamento" ──► GET /chamados/{id}/acompanhamento
  estado não terminal → repete (polling, ETag, backoff)
  ENCERRADO           → para
```

Fluxo 5 — Integração futura com órgãos

```
EncaminhamentoService        ──► EncaminhamentoGateway ◄── Simulado (MVP) | Real (futuro)
DeterminacaoDeCentralService ──► JurisdicaoGateway     ◄── Mock (MVP)     | Real (futuro)
retorno de status ──────────────► mesmo caminho de atualização de estado
```

16. API REST

Base `/api/v1`, JSON, datas ISO-8601 em UTC, `Cache-Control: no-store` nas respostas com dados pessoais. O **contrato REST é a fronteira** entre os clientes (Web e Android futuro) e o backend.

Método e rota	Objetivo	Auth
POST `/api/v1/auth/login`	Autenticar por `identificador` (CPF ou e-mail) + senha	Pública
POST `/api/v1/auth/refresh`	Renovar sessão	Refresh
POST `/api/v1/auth/logout`	Encerrar sessão (revoga o refresh)	Sim
POST `/api/v1/usuarios`	Cadastro	Pública
GET `/api/v1/usuarios/me`	Dados do Perfil	Sim
GET `/api/v1/ocorrencias?orgao=`	Catálogo ativo do órgão	Sim
POST `/api/v1/chamados`	Registrar solicitação (SAMU, Bombeiros, PM)	Sim
POST `/api/v1/chamados/emergencia-imediata`	Registrar Emergência Imediata	Sim
GET `/api/v1/chamados`	Histórico (paginado, do usuário)	Sim
GET `/api/v1/chamados/{id}`	Detalhes da solicitação	Sim
GET `/api/v1/chamados/{id}/acompanhamento`	Estado atual e transições	Sim
GET `/api/v1/contatos-confianca`	Listar contatos	Sim
POST `/api/v1/contatos-confianca`	Adicionar contato (até 5)	Sim
PUT `/api/v1/contatos-confianca/{id}`	Editar contato	Sim
DELETE `/api/v1/contatos-confianca/{id}`	Remover contato	Sim
POST `/api/v1/contatos-confianca/{id}/compartilhamentos`	Registrar `COMPARTILHAMENTO_INICIADO`	Sim
GET `/api/v1/configuracoes/cliente`	Limiar de precisão (100 m) e limites de texto	Sim

Contratos relevantes

- `POST /chamados`: corpo com órgão, ocorrência, detalhe opcional e localização. Header `Idempotency-Key`. Retorna 201 no primeiro registro e 200 em replay.
- `POST /chamados/emergencia-imediata`: corpo com detalhe opcional e localização, **sem órgão e sem ocorrência**. Mesmo mecanismo de idempotência.

- `GET /chamados/{id}` devolve: código da simulação, modalidade, órgão, ocorrência (nula na EI), localização, `baixaPrecisao`, horário, detalhe adicional, estado e `modoEncaminhamento: SIMULADO`. O texto "Emergência Imediata → Polícia Militar" é montado pela UI.
- `POST /contatos-confianca/{id}/compartilhamentos`: o cliente chama de forma **não bloqueante**. Se falhar, o compartilhamento nativo segue. O backend grava só o evento `COMPARTILHAMENTO_INICIADO` (ator, instante, contato, canal opcional), **sem conteúdo da mensagem e sem localização**, e **nunca** afirma enviado, entregue, recebido ou lido.
- `GET /configuracoes/cliente` devolve o limiar de baixa precisão (100 m, configurável no backend) e os limites de texto, para o frontend não duplicar a regra.
- Histórico: ordenado do mais recente ao mais antigo, paginado, sempre escopado ao usuário autenticado. Cada chamado tem estado próprio.

Erros: `application/problem+json`

Estrutura com `status`, `code` estável legível por máquina, `detail` seguro (sem detalhes internos), `errors[]` por campo e `correlationId`. Nunca há stack trace.

Situação	HTTP	`code` (exemplos)
Validação	400	`VALIDACAO_INVALIDA`
Não autenticado	401	`NAO_AUTENTICADO`, `CREDENCIAIS_INVALIDAS`, `TOKEN_EXPIRADO`
Conta inativa (somente com senha correta)	403	`CONTA_INATIVA`
Inexistente ou pertencente a outro usuário	404	`RECURSO_NAO_ENCONTRADO`
Conflito de cadastro ou de idempotência	409	`CADASTRO_JA_EXISTENTE`, `IDEMPOTENCIA_DIVERGENTE`
Regra de negócio	422	`LIMITE_CONTATOS_EXCEDIDO`
Limite de requisições	429	`LIMITE_DE_REQUISICOES`
Indisponibilidade / erro inesperado	503 / 500	`SERVICO_INDISPONIVEL` / `ERRO_INTERNO`

Emergência Imediata: endpoint próprio

O contrato é diferente do fluxo normal: sempre PM, sem ocorrência, modalidade própria e confirmação específica. **Internamente** usa o mesmo agregado `Chamado`, a mesma lógica de negócio e o mesmo `AberturaDeChamadoService`, com uma factory própria no agregado. Não há duplicação de domínio.

17. Idempotência

O envio de uma emergência é operação crítica e deve ser idempotente, **sem UUID** e respeitando a decisão global de identificadores `Long`.

- A chave é gerada **no cliente**, é **opaca**, **não é UUID**, não é identificador da entidade nem chave primária, e é **associada ao usuário**.
- É **reutilizada nos retries**, com o **payload congelado** na primeira tentativa (incluindo a localização).
- **Mesma chave + mesmo payload = replay**: devolve o chamado original (200).
- **Mesma chave + payload diferente = conflito**: 409 `IDEMPOTENCIA_DIVERGENTE`.
- Garantia final por unicidade `(usuario_id, chave_idempotencia)`. Em concorrência, a requisição perdedora cai na constraint e responde como replay.
- **Sem TTL** no banco: a chave vive com o chamado, sem tabela extra nem job de limpeza.
- Não há heurística de "duplicata parecida": dois chamados legítimos iguais são possíveis (múltiplos chamados simultâneos).
- No cliente, o botão é desabilitado após o toque e os retries por timeout ou rede incerta reutilizam chave e payload.

- O **formato técnico exato** da chave (tamanho, codificação, validação de charset) fica para a implementação.

Descartado: chave emitida pelo servidor, que exige um round-trip antes de pedir ajuda e falha justamente com conexão ruim.

18. Simulação

A simulação **não** fica no Controller, no frontend nem no agregado Chamado. Fica no módulo `encaminhamento`, na Infrastructure, como implementação do `EncaminhamentoGateway` (`EncaminhamentoSimuladoAdapter`) e de um job de aplicação.

```
Chamado registrado
       ↓
Determinação de Central
       ↓
Encaminhamento SIMULADO
       ↓
job persistente
       ↓
CRIADO → ENCAMINHADO_SIMULADO → ENCERRADO
```

- **Persistente:** o estado e o agendamento (`proxima_transicao_em`) ficam no banco e sobrevivem a reinício do backend. Não há timers puramente em memória.
- Um job `@Scheduled` processa os encaminhamentos vencidos e chama o `EncaminhamentoService`. Cada transição é gravada no Chamado e em `chamado_transicao`, em transação por chamado, de forma **idempotente**.
- **Reconciliação:** o mesmo job recolhe chamados `CRIADO` sem encaminhamento, caso o evento pós-commit se perca.
- As durações são **parâmetros configuráveis**. Os 8 e 20 segundos do protótipo não são regra de negócio.
- Sem WebSocket nem SSE no MVP. O cliente usa polling.
- **Simetria com o real:** o caminho que atualiza o estado é o mesmo que um polling ou webhook de um órgão real chamaria. Substituir a simulação não altera o domínio.
- As regras de falha e reprocessamento de integração real permanecem **suspensas** no MVP.
- A implementação detalhada do job (frequência, lote, bloqueio para várias instâncias) fica para a implementação.

19. Jurisdição

- `JurisdicaoGateway` fica no **Domain** do módulo `jurisdicao` (decisão K1). O domínio não conhece API externa, fornecedor, protocolo nem serviço de geocodificação.
- No MVP é usado um **mock** com jurisdição e central **explicitamente fictícias**. Nenhuma integração oficial é inventada.
- **Jurisdicao ≠ CentralAtendimento.** Jurisdição é o recorte territorial; Central é a unidade operacional, resolvida depois pelo `DeterminacaoDeCentralService`.
- Não há tabelas para esses conceitos no MVP. O `Encaminhamento` guarda referências.
- A jurisdição serve ao **encaminhamento**, não ao **registro**: o registro funciona mesmo que a consulta falhe.
- Substituição futura: troca-se somente o adapter. O contrato permanece.
- Dados que uma integração real poderia exigir (a definir com cada órgão, sem inventar): coordenadas, precisão, timestamp, órgão, classificação da ocorrência, município/UF, identificador da central, protocolo externo e canal de retorno.

20. Segurança

Autenticação

- Login por um único campo `identificador`: se contém `@`, é e-mail; caso contrário, CPF normalizado e validado. Não há `Credencial` nem `TipoIdentificador`.
- Senha armazenada **somente como** `senhaHash`, com hash forte (BCrypt ou Argon2id). Nunca em texto puro.
- Credenciais inválidas: resposta genérica (`CREDENCIAIS_INVALIDAS`, "Dados incorretos"), com tempo de resposta equalizado. `CONTA_INATIVA` **somente** quando a senha está correta, reconciliando a mensagem exigida para conta inativa com a não revelação de existência de CPF ou e-mail.
- Cadastro com CPF ou e-mail já existente retorna erro específico no campo (decisão aprovada na Etapa 2), mitigado por rate limiting.
- Sem bloqueio rígido de conta; throttling progressivo por IP e identificador.

Tokens

Item	Decisão
Access token	JWT assinado, de curta duração (cerca de 15 min), claims mínimos (`sub` = id do usuário), sem dado pessoal.
Refresh token	Opaco, **rotativo**, guardado somente como hash, com detecção de reuso. Validade longa para não deslogar o usuário no meio de uma emergência.
Web	Access token somente em memória. Refresh em cookie `httpOnly; Secure; SameSite`, com proteção CSRF. **Nenhum token em localStorage.**
Android futuro	Refresh no armazenamento seguro do dispositivo. Detalhes adiados.
Logout	Revoga o refresh token.

Autorização e demais requisitos

- **Default-deny:** todo `/api/**` exige autenticação, exceto login, refresh e cadastro.
- O `usuarioId` vem **sempre do token**, nunca do frontend. Toda consulta é escopada por ele.
- Recurso pertencente a outro usuário retorna **404** (não 403), para não revelar existência.
- TLS obrigatório e HSTS; cabeçalhos de segurança (CSP, `nosniff`, `frame-ancestors`, `Referrer-Policy`); CORS por allowlist exata, nunca `*`.
- Rate limiting por endpoint. No `POST /chamados` é apenas uma **válvula de segurança**, folgada, nunca regra de negócio.
- Queries parametrizadas, DTOs sem campos sensíveis de entrada, segredos fora do repositório, usuário de banco com privilégio mínimo.
- Sessão expirada: o frontend exibe banner e redireciona ao Login com aviso.

21. Frontend

React, TypeScript, Vite, React Router, TanStack Query, estrutura por features, coerente com as **13 telas e 1 modal** aprovados nas Etapas 3 e 3.1.

```
src/
 ├─ app/        providers, rotas, layouts, ProtectedRoute, error boundary
 ├─ features/   auth, home, solicitacao, acompanhamento, historico, contatos, perfil
 ├─ shared/     ui (componentes), api, hooks, a11y, lib
 └─ platform/   fronteira para capacidades do dispositivo (detalhes na Etapa 7)
```

- **Telas:** Splash, Login, Cadastro, Home, Seleção de ocorrência, Resumo, Solicitação registrada, Acompanhamento, Avisar contato, Gerenciar contatos, Histórico, Detalhes da solicitação, Perfil; mais o modal da Emergência Imediata, que abre sobre a Home sem navegar.

- **Dois layouts:** com navegação inferior (Início, Histórico, Perfil) e de emergência (tela cheia, sem navegação inferior).
- **Componentes** mapeados 1:1 com a Etapa 3: `AppButton`, `OrgaoActionCard`, `ImmediateEmergencyButton`, `TrustedContactCard`, `OccurrenceCard`, `LocationStatusIndicator`, `SummaryField`, `StatusTimeline`, `SimulationBanner`, `ConfirmationModal`, `ContactListItem`, `BottomNav`, `ErrorBanner`.
- **Estado:** servidor com TanStack Query (histórico, detalhe, acompanhamento, catálogo, contatos); global somente a sessão; local com useState/useReducer.
- **Cliente HTTP único:** anexa o access token e o `X-Correlation-Id`; em 401 faz refresh single-flight e repete uma vez; converte problem+json em erro tipado; se o refresh falha, banner de sessão expirada e Login.
- **Proteção de rotas** é conveniência de UX. A proteção real é do backend.
- **Acompanhamento por polling** (para ao encerrar, pausa com a aba oculta, ETag, backoff em erro). No Histórico, **uma** requisição de lista enquanto houver item não encerrado.
- `SimulationBanner` exibido quando a resposta traz `modoEncaminhamento = SIMULADO`.
- **Acessibilidade:** WCAG 2.1 AA, alvos de toque de pelo menos 44 px, nada só por cor, `aria-live` para mudanças de status, linguagem simples.

Máquina de estados da localização (permanece no frontend)

Estado	Origem	Comportamento
obtendo	Chamada em curso	Indicador “Obtendo localização...”
obtida	Sucesso, precisão ≤ limiar	Envio habilitado
baixa precisão	Sucesso, precisão > limiar (100 m)	**Não bloqueia.** Aviso visual e “Melhorar localização” opcional. Na EI, aviso breve e segue sozinho, sem segunda confirmação
negada	Permissão negada	Envio desabilitado, com orientação
indisponível	Posição indisponível	Envio desabilitado, com orientação
erro / tentar novamente	Timeout ou falha	Retry

O **backend continua sendo a autoridade** para validações, limiar de precisão, regras de negócio, autorização e idempotência. O limiar vem de `GET /configuracoes/cliente`. O backend recalcula `baixaPrecisao` no registro e a persiste, para o Histórico refletir o que o usuário viu na época. O backend não rejeita localização por idade do timestamp, para não bloquear uma emergência.

Envio: no fluxo normal, a captura começa ao entrar no Resumo, e “Enviar mensagem” fica habilitado quando a localização está obtida ou com baixa precisão. Na EI, ENVIAR AGORA dispara a captura em segundo plano. Cancelar durante a captura aborta o envio e nada é registrado. Falha de rede: retry limitado com a mesma chave e o mesmo payload; depois, “Não conseguimos confirmar o envio. Verifique o Histórico”.

Contato de confiança: fronteira

O SINAL persiste	O dispositivo / app externo faz
Nome e telefone do contato (até 5, invariante do agregado de contatos).	Abrir SMS, WhatsApp ou Telegram, montar a mensagem, incluir a localização e enviar.
Evento de auditoria `COMPARTILHAMENTO_INICIADO` (sem conteúdo e sem localização).	O envio em si. O SINAL não sabe se foi enviado, entregue, recebido ou lido.

A interface diz “Mensagem pronta para envio”, nunca “enviada”. A mensagem opcional e a localização do aviso ao contato são tratadas no cliente e **não são persistidas**. Não existe endpoint de notificação nem entidade “mensagem entregue”.

22. Android futuro

- O Android será **outro cliente da mesma API**, posterior ao MVP Web, empacotando o frontend (por exemplo com Capacitor).
- A arquitetura interna do backend **não** é desenhada especificamente para Android. O **contrato REST** é a fronteira.
- A API aceita refresh por body e não depende exclusivamente de cookies. As origens do Capacitor entram na allowlist de CORS quando necessário.
- A pasta `platform/` do frontend isola capacidades do dispositivo (localização, compartilhamento, armazenamento seguro, háptico). O desenho concreto fica para a Etapa 7.
- O **acionamento físico de SOS** (botão de volume ou similar) é **futuro** e não é requisito desta etapa. A regra estável permanece: destino sempre PM, com confirmação explícita.

```
React Web ──► Spring Boot API ◄── Android / Capacitor (futuro)
```

23. LGPD

A LGPD é tratada como **requisito arquitetural**, sem inventar requisitos legais além do necessário.

- **Minimização:** localização somente quando necessária, no momento do chamado; sem rastreamento contínuo.
- **CPF mascarado** em respostas e logs. Dados de terceiros nos contatos (nome e telefone) sob o escopo do titular.
- O **detalhe adicional** e as categorias do SAMU podem conter dados relacionados à **saúde**: nunca entram em logs ou métricas.
- **Nunca registrar em logs:** senha, tokens, CPF completo, telefone, coordenadas exatas, detalhe adicional, conteúdo de compartilhamento.
- Dados atrelados a `usuario_id`, o que facilita qualquer política futura de exclusão.
- Criptografia de campo (CPF, detalhe adicional) **adiada**: conflita com a busca por CPF e exigiria índice cego.
- A **política definitiva de retenção e exclusão** permanece adiada. Nenhuma política foi inventada.

24. Observabilidade

- **Logs** em JSON estruturado de criação, simulação e mudança de estado. `X-Correlation-Id` aceito ou gerado, devolvido na resposta e presente no problem+json.
- **Métricas** (Actuator + Micrometer, sem alta cardinalidade): chamados por modalidade e órgão, replays de idempotência, transições da simulação, atraso do job, falhas de login, latência e erros HTTP.
- **Health** de liveness e readiness, com endpoints de gerenciamento restritos.
- **Auditoria** (`auditoria_evento`), obrigatória e sem dados sensíveis desnecessários: criação do chamado, transições da simulação e `COMPARTILHAMENTO_INICIADO`.
- Sem stack de observabilidade externa no MVP. Sem infraestrutura de produção nesta etapa.

25. Testes

Nível	O que cobre
Domínio (unitário, sem Spring)	Invariantes e transições do Chamado, `Localizacao`, limite de 5 contatos, EI ⇒ PM
Serviços de aplicação	Idempotência, propriedade, orquestração, com fakes dos ports
Persistência	Adapters e mappers, `@DataJpaTest` com MySQL (Testcontainers), migrações
API	Contratos, problem+json, mass assignment, rotas protegidas
Segurança	Default-deny, 404 em recurso alheio, erros de login genéricos, refresh
Integração	Fluxo de registro até `ENCERRADO` com a simulação
Contrato anti-falsa-afirmação	Nenhum texto ou estado afirma “central recebeu”, “equipe a caminho”, “polícia acionada” ou “em atendimento”

ArchUnit (uso proporcional: 4 regras)

- 1. `domain` não depende de Spring, JPA nem Jackson.
- 2. `domain` e `application` não dependem de `presentation`.
- 3. `presentation` não acessa `infrastructure` diretamente.
- 4. Módulos só se acessam pela API pública e não possuem ciclos.

Não há bateria extensa de testes arquiteturais. Se o custo não valer o benefício, a regra 4 é a primeira candidata a ser descartada.

26. Microsserviços e evolução futura

O SINAL é um **monólito modular**. Não há microsserviços no MVP, nem Kafka, RabbitMQ, WebSocket, SSE ou Redis sem necessidade.

A separação por módulos, a comunicação por eventos e as referências por ID criam uma base mais favorável a uma eventual extração futura de módulos para microsserviços, caso isso seja necessário.

Entretanto, essa possibilidade **não deve orientar decisões que aumentem a complexidade do MVP**. A arquitetura deve ser avaliada prioritariamente pela clareza, testabilidade, manutenibilidade e facilidade de evolução do monólito modular atual.

O SINAL não está sendo construído como um conjunto de microsserviços antecipados. A eventual extração futura é uma possibilidade arquitetural, **não um objetivo do MVP**.

27. Decisões aprovadas

ID	Decisão	Origem
D1	Monólito modular, sem microserviços	Etapas 1 e 4
D2	Java + Spring Boot, API REST	Etapa 1
D3	React + TypeScript, Web responsiva, mobile-first	Etapa 1
D4	MySQL	Etapa 1
D5	`Long` como identificador; sem UUID como identificador de entidade	Etapa 1
D6	`Instant` para timestamps, persistidos em UTC; apresentação em America/Sao_Paulo	Etapa 1
D7	`OrgaoSeguranca` como enum (POLICIA_MILITAR, CORPO_DE_BOMBEIROS, SAMU)	Etapa 1
D8	`Localizacao` como Value Object imutável do Chamado	Etapa 1
D9	`Chamado` como Aggregate Root	Etapa 1
D10	Sem `Credencial` e sem `TipoIdentificador`; login por CPF ou e-mail; `senhaHash`	Etapas 1 e 2
D11	Baixa precisão não bloqueia o envio (PM, SAMU, Bombeiros, EI); limiar de 100 m, configurável no backend	Etapa 3.1 e 4
D12	Emergência Imediata como modalidade de Chamado, direcionada à PM, com endpoint próprio e sem ocorrência	Etapas 3 e 4
D13	Múltiplos Chamados simultâneos por usuário	Etapa 3.1
D14	Histórico com consulta por ID; tela Detalhes; acompanhamento por chamado	Etapa 3.1
D15	Simulação separada da integração real; Ports/Adapters; `JurisdicaoGateway` e `EncaminhamentoGateway`	Etapas 1 e 4
D16	Até 5 contatos; nenhuma confirmação falsa de entrega; sem “Em Atendimento”	Etapas 2 e 3
D17	Android futuro como cliente da mesma API; REST é a fronteira	Etapa 4
D18	Quatro camadas (Domain, Application, Presentation, Infrastructure) aplicadas somente onde há responsabilidade real	Etapa 4
D19	Domain Entity ≠ JPA Entity nos agregados com comportamento	Etapa 4
D20	Catálogo de ocorrências persistido, não enum, com “Outro / Não sei informar” por órgão	Etapa 4
D21	Idempotência por chave opaca do cliente, não UUID, única por (usuário, chave), payload congelado, sem TTL	Etapa 4
D22	Histórico de estados em tabela append-only	Etapa 4
D23	Simulação persistente, fora de Controller, frontend e agregado, com job `@Scheduled` e reconciliação	Etapa 4
D24	JWT curto + refresh token rotativo; sem tokens em localStorage	Etapa 4
D25	Acompanhamento por polling; sem WebSocket/SSE no MVP	Etapa 4
D26	Erros em `application/problem+json` com `code` estável e `correlationId`	Etapa 4
D27	Contatos como agregado por usuário, com limite de 5 como invariante	Etapa 4
D28	Auditoria `COMPARTILHAMENTO_INICIADO` obrigatória; falha não bloqueia o fluxo	Etapa 4
D29	Estados internos `CRIADO`, `ENCAMINHADO_SIMULADO`, `ENCERRADO`; textos de UI aprovados	Etapas 2, 3 e 4
D30	Detalhe adicional opcional, texto simples, máximo de 500 caracteres	Etapa 4
D31	`Encaminhamento` permanece separado de `Chamado`	Etapa 4
D32	Cancelamento fora do MVP: sem endpoint, sem estado `CANCELADO`, sem tela	Etapas 3.1 e 4
K1	`JurisdicaoGateway` e `EncaminhamentoGateway` permanecem no **Domain**	Etapa 4
K2	`EncaminhamentoService` e `DeterminacaoDeCentralService` são **Application Services**	Etapa 4
D33	Auditoria (`auditoria_evento`) obrigatória; nunca registrar dados sensíveis em logs	Etapas 2 e 4
D34	Flyway para migrações; observabilidade básica; LGPD como requisito arquitetural	Etapa 4

28. Decisões descartadas

Tema	Descartado	Motivo
Arquitetura	Reduzir a Controller → Service → Repository → Entity	Decisão de manter separação clara entre regras de negócio, casos de uso, entrada HTTP, persistência e integrações.
Camadas	Quatro camadas obrigatórias em todo módulo	Cerimônia. Adotado o subconjunto necessário por módulo.
Persistência	Domínio anotado com JPA (`@Embeddable` no domínio)	Quebra a independência do domínio.
Persistência	Mapper e agregado para catálogo, registros técnicos e consultas	Sem comportamento de domínio.
Application	Interface por serviço de aplicação	Sem substituição nem ganho de teste.
Ocorrências	Enum	Evolução, ordem, ativação e códigos estáveis exigem tabela.
Idempotência	UUID	Contraria a decisão de `Long`.
Idempotência	Chave emitida pelo servidor	Round-trip extra antes de pedir ajuda.
Idempotência	Heurística de duplicata parecida	Bloquearia chamados legítimos.
Simulação	Estado calculado na leitura	Sem efeito persistente e sem transições gravadas.
Simulação	Timers em memória	Perdem estado ao reiniciar.
Acompanhamento	WebSocket / SSE	Desproporcional ao MVP.
Sessão	Tokens em localStorage; sessão de servidor	Exposição a XSS; menor aderência ao Android futuro.
Contato	Endpoint de notificação; mensagem “entregue”	Proibido: nenhuma falsa confirmação.
Eventos	Event sourcing	Desnecessário e proibido no briefing.
Estados	“Em Atendimento”; `CANCELADO`	Sugerem atendimento real / fora do escopo do MVP.
Módulos	Um módulo por conceito (usuário, histórico, localização)	Fragmentação artificial.
Microsserviços	Como objetivo do MVP	Apenas consequência possível, não requisito.
Testes	Bateria extensa de ArchUnit	Burocracia.

29. Decisões adiadas

Para a Etapa 5 (Modelo de Domínio)

- nomes finais das classes;
- enums;
- modelagem detalhada do agregado Chamado;
- Value Objects;
- invariantes detalhadas;
- máquina de estados;
- modelagem de Encaminhamento;
- modelagem de Jurisdicao;
- modelagem de CentralAtendimento;
- eventual Domain Service;
- agregado de contatos;
- nomenclatura definitiva; formato do código da simulação (`#SIM-` + id).

Para a implementação

- formato técnico exato da `Idempotency-Key`;
- convenção Port/Gateway/UseCase/Service;
- MapStruct ou mapper manual;
- frequência e lote do job;
- locking do job;
- rate limiting (valores);
- duração da simulação;
- detalhes de cookies e CSRF;
- tempos exatos de tokens;
- nomes finais de tabelas e colunas.

Para etapas futuras

- política de retenção e exclusão (LGPD);
- criptografia de campo;
- plataforma Android (`platform/`, Etapas 7 e 9);
- integração real com órgãos e dados necessários;
- retry e falha de integração real;
- infraestrutura de produção.

30. Riscos

Risco	Mitigação
Burocracia crescer com a estrutura de camadas	Teste de necessidade (seções 4 e 13) e subconjuntos de camadas por módulo.
Mapper domínio ↔ JPA gerar bugs	Testes de persistência dedicados aos adapters.
Sessão expirar no meio de uma emergência	Refresh longo e rotativo. Se falhar, banner e Login.
Retry duplicar solicitação	Idempotência com payload congelado e constraint única.
Job de simulação parado	Métrica de atraso e reconciliação.
Evento pós-commit perdido	Reconciliação pelo job.
Cadastro revelar existência de CPF ou e-mail	Decisão aprovada (Etapa 2). Rate limiting.
Dado de saúde em texto livre	Não logar, acesso restrito, avaliar cifra de campo no futuro.
Cifrar CPF conflitar com unicidade	Adiado; índice cego se necessário.
Rate limiting barrar emergência real	Limite folgado, apenas válvula de segurança.
Divergência do limiar de precisão entre cliente e backend	Limiar único via `/configuracoes/cliente`.
Texto sugerir atendimento real	Teste de contrato anti-falsa-afirmação.
Falha ao registrar `COMPARTILHAMENTO_INICIADO`	Não bloqueia o fluxo; perda tolerável da auditoria.
Fronteiras de módulo frouxas sem verificação	ArchUnit proporcional (regras 1 a 4).

31. Critérios de aceitação

- Domain independente de Spring, JPA, HTTP e JSON, verificado por teste arquitetural.
- Presentation sem regra de negócio e sem acesso a repositório. Sem mass assignment.
- Cada abstração justifica-se pelo teste de necessidade (substituição, isolamento do domínio ou testabilidade).

- Simulação persistente, fora de Controller, frontend e agregado, com reconciliação.
- Idempotência conforme a seção 17. Estados conforme a seção 8.
- Nenhum texto ou estado afirma atendimento real; nenhuma mensagem é dada como enviada, entregue, recebida ou lida.
- Cancelamento, `CANCELADO`, “Em Atendimento”, WebSocket/SSE e microsserviços ausentes do MVP.
- Recurso de outro usuário retorna 404. `usuarioId` nunca vem do cliente. Nenhum token em localStorage.
- Android registrado como cliente da mesma API, sem influenciar o desenho interno do backend.
- Nenhum código, migration, UML detalhado ou integração real produzido nesta etapa.

32. Validação final da Etapa 4

Item	Definido	Adiado	Risco relevante
A. Arquitetura geral	Monólito modular; quatro camadas onde há responsabilidade real; Spring onde pertence; teste de necessidade	—	Burocracia (mitigada)
B. Módulos	7 módulos com subconjuntos de camadas	Nomes finais de pacotes	Fronteiras frouxas sem ArchUnit
C. Domain	Java puro; Chamado AR; Localizacao VO; não anêmico; gateways de negócio no domínio (K1)	Modelagem fina (Etapa 5)	Agregado anêmico se orquestração vazar
D. Application	AberturaDeChamadoService, EncaminhamentoService, DeterminacaoDeCentralService como serviços de aplicação (K2)	Convenção de nomes	Regras migrarem para o serviço
E. Presentation	Controllers, DTOs, validação sintática, sem mass assignment	—	Baixo
F. Infrastructure	JPA, Spring Data, adapters, segurança, simulação, mock, observabilidade	—	Baixo
G. Persistência	Domain ≠ JPA onde há comportamento; repositório em 3 peças só para agregados; simplificações do catálogo, consultas e registros técnicos; Flyway	Nomes de tabelas e colunas	Bugs de mapper (testes de adapters)
H. Segurança	JWT curto, refresh rotativo, default-deny, 404 em recurso alheio, sem localStorage	Cookies/CSRF e tempos	Sessão longa em dispositivo perdido
I. Idempotência	Chave opaca do cliente; payload congelado; replay e conflito 409; sem TTL	Formato técnico	Retry com payload alterado (tratado)
J. Simulação	Persistente, `@Scheduled`, reconciliação, fora de controller, front e agregado	Frequência, lote e locking	Job parado (métrica)
K. Jurisdição	Gateway no domínio; mock fictício; sem tabelas	Dados de integração real	Baixo
L. Contatos	Agregado por usuário, até 5; sem envio; `COMPARTILHAMENTO_INICIADO`	Forma final do agregado (Etapa 5)	Falha da auditoria não bloqueia
M. Frontend	Stack e features; 13 telas + 1 modal; máquina de localização no front	`platform/` (Etapa 7)	Baixo
N. API	Lista fechada; problem+json; EI separada; compartilhamentos	—	Baixo
O. Android futuro	Cliente da mesma API; REST como fronteira	`platform/`, Etapas 7 e 9	Baixo
P. Testes	Domínio, serviços, persistência, API, segurança, integração, anti-falsa-afirmação, ArchUnit com 4 regras	—	Cobertura insuficiente se ArchUnit cortado
Q. LGPD	Minimização, CPF mascarado, nunca logar dados sensíveis	Retenção e exclusão; cifra de campo	Dado de saúde em texto livre
R. Observabilidade	Logs JSON, correlação, métricas essenciais, health, auditoria obrigatória	—	Baixo

Checklist

- Arquitetura Domain / Application / Presentation / Infrastructure confirmada, com gateways de negócio no Domain (K1).
- Application Services confirmados para EncaminhamentoService e DeterminacaoDeCentralService (K2).
- Estrutura de módulos com subconjuntos de camadas, sem camadas vazias.
- Domain Entity ≠ JPA Entity somente onde há comportamento; simplificações aplicadas.

- Cancelamento, `CANCELADO`, “Em Atendimento”, WebSocket/SSE e microsserviços fora do MVP.
- Idempotência, baixa precisão (100 m), EI separada, catálogo, detalhe (500) e estados conforme este documento.
- `COMPARTILHAMENTO_INICIADO` auditado sem afirmar envio ou entrega.
- Segurança, LGPD e observabilidade conforme as seções 20, 23 e 24.
- Decisões adiadas registradas na seção 29.

33. Próxima etapa

A próxima etapa é a **Etapa 5 — Modelo de Domínio**, que tratará, entre outros: agregados; entidades; Value Objects; enums; invariantes; relacionamento entre agregados; ciclo de vida do Chamado; máquina de estados; Encaminhamento; Jurisdicao; CentralAtendimento; catálogo de ocorrências; contratos relevantes; e o modelo conceitual.

A Etapa 5 recebe desta etapa as fronteiras, os contratos de módulo e as decisões adiadas listadas na seção 29. **Não foi iniciada.**

Etapa 4 — Arquitetura Técnica: CONCLUÍDA E APROVADA.

A Etapa 5 será iniciada somente após a validação deste documento final e seguirá para o Modelo de Domínio.